

实验一 Quartus 软件与状态机第一部分

一、实验目的

- (1) 复习巩固 Quartus 软件流程;
- (2) 掌握 Verilog HDL、VHDL、BDF 图形输入之间的本质关联;
- (3) 学习 Verilog HDL、VHDL 的基本语法。

二、实验原理与步骤

1. 在电脑上安装 Quartus 软件环境。

要求软件版本不低于 18.1 Lite, 带 Modelsim_Altera;

器件包支持: Cyclone IV、Cyclone V、MAX V、MAX 10。

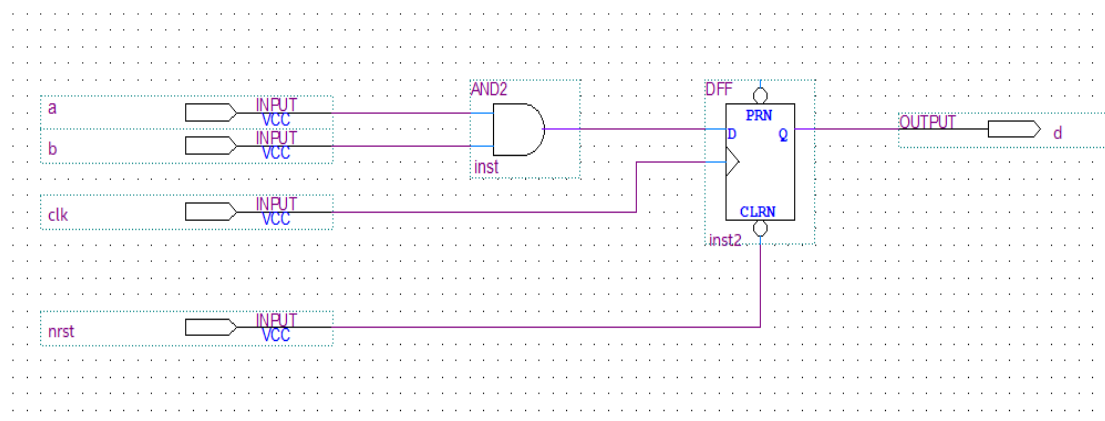
2. 新建工程 hello, 新建 BDF 图形, 完成以下设计:

信号 a&b 作为输入 D 触发器输入, 信号 clk 为时钟信号,

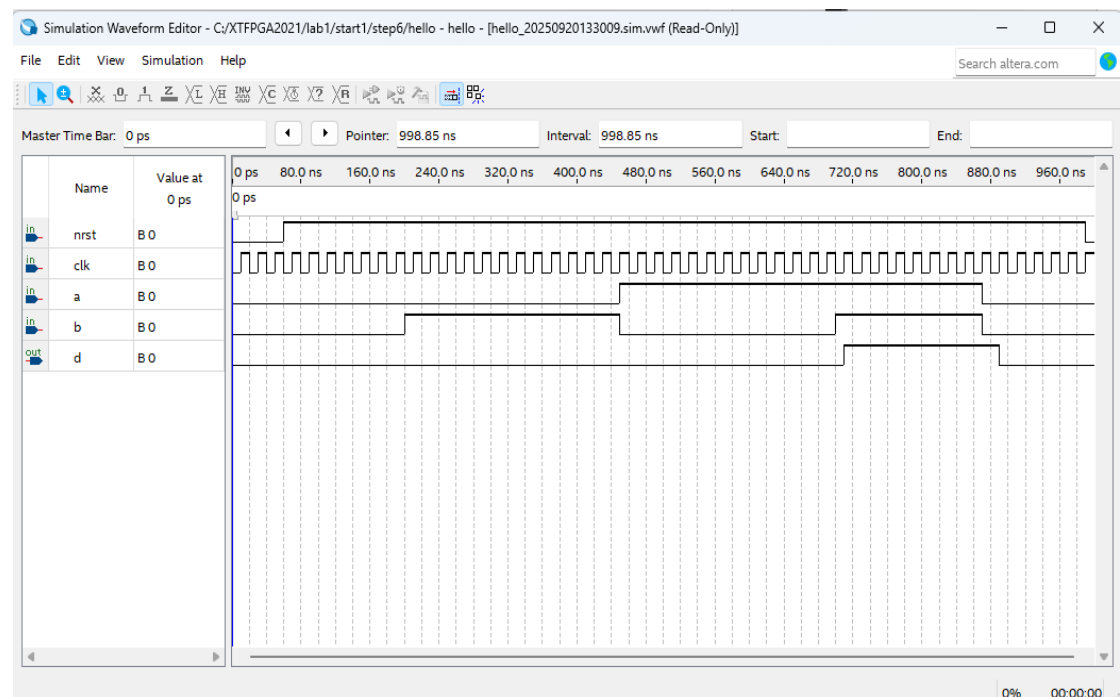
信号 nrst 为复位信号, 信号 d 为输出。

在 processing 中进行编译, 新建 waveform 文件, 在 node finder 检索时选择 Design Entry 添加信号, 并在 simulation 中进行功能仿真和时序仿真。

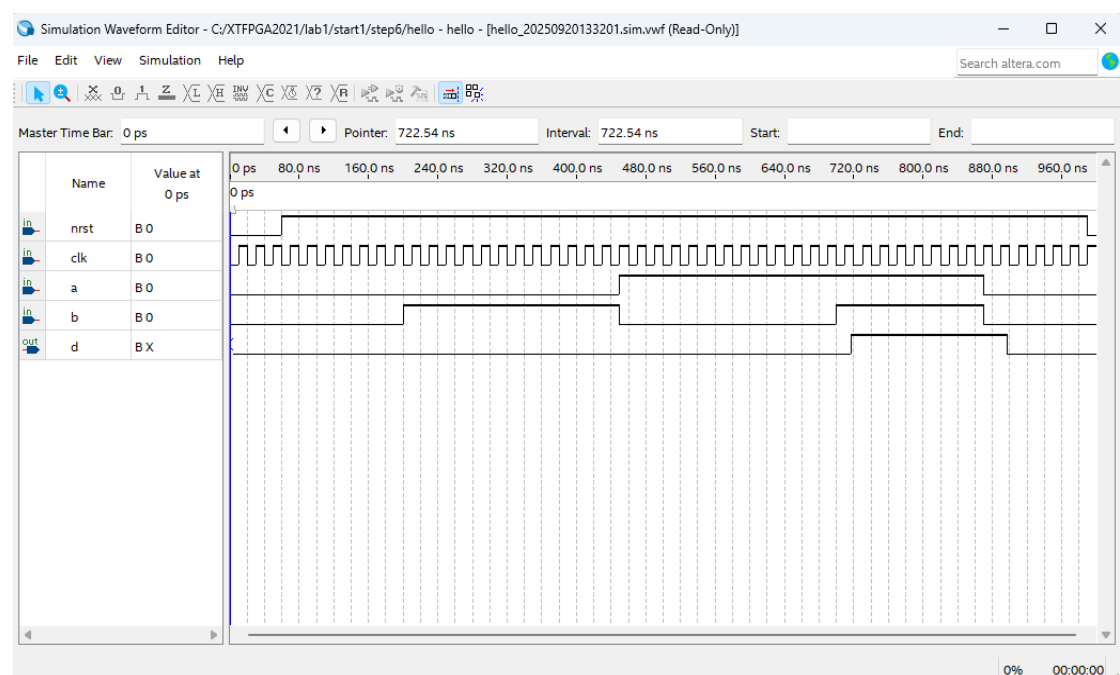
BDF 图形:



功能仿真图形：



时序仿真图形：



3. 利用 Insert templates 将 BDF 图形转化为 verilog 文件,并进行编译和仿真。

代码如下：

```
// Quartus Prime Verilog Template
```

```

module hellov

(
    input a,

    input b,

    input clk,

    input nrst,

    output reg d
);//定义结构体输入和输出

always @ (negedge nrst or posedge clk)

begin

    // 复位信号 nrst 由 1 变 0 时重置触发器

    if (!nrst)

        begin

            d <= 1'b0;// 将触发器置 0

        end

    // 不触发复位重置时，在时钟上升沿更新寄存器输出为 a&b

    else

        begin

            d <= a&b;

        end

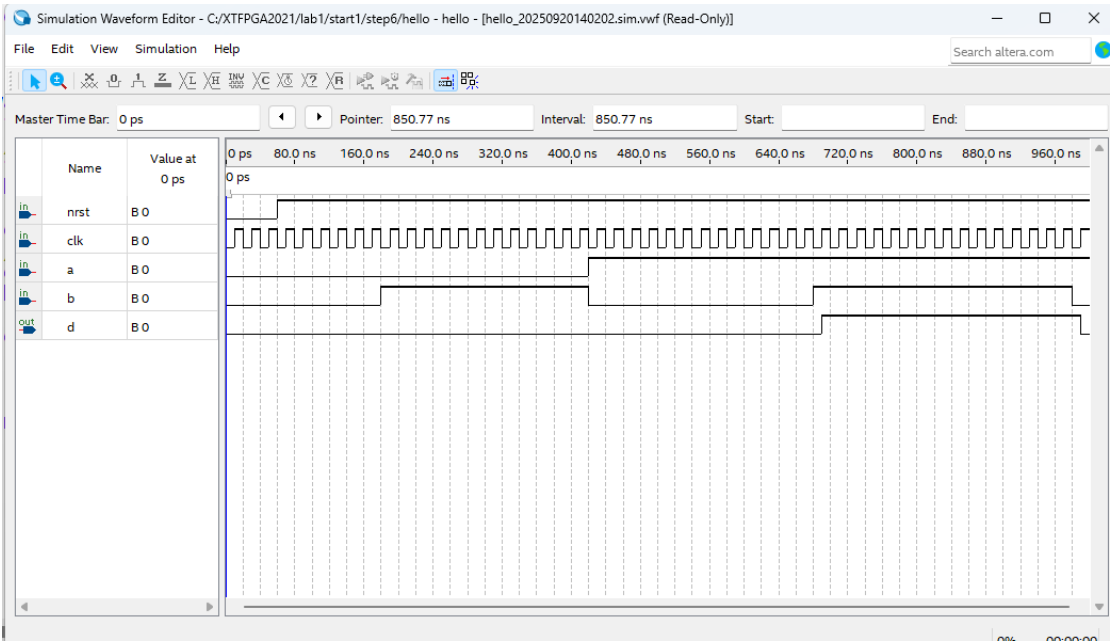
    end

end

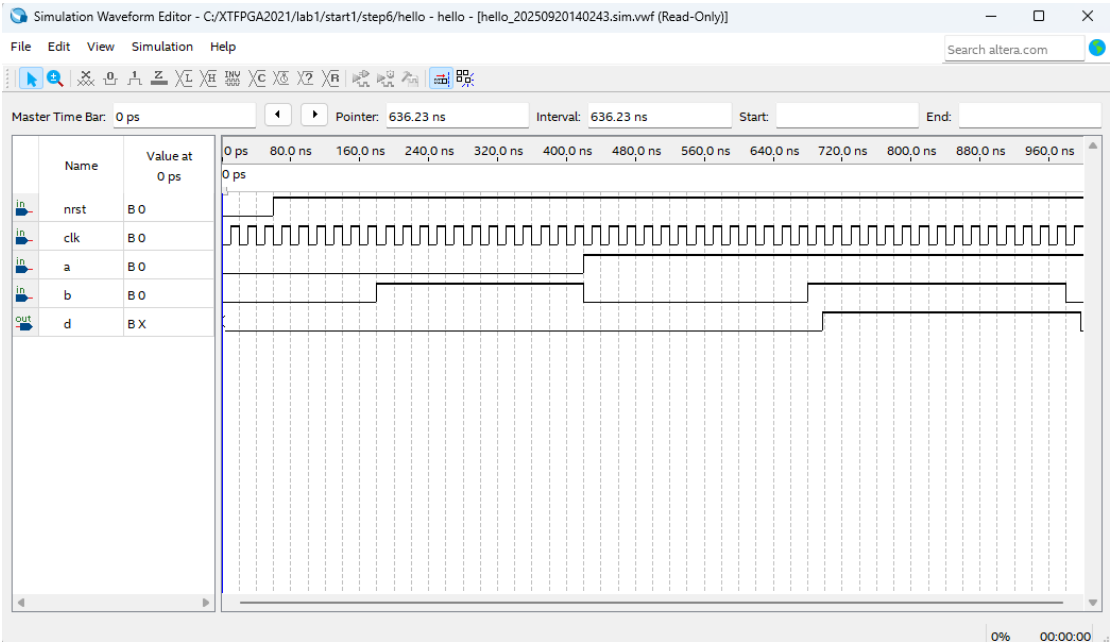
endmodule

```

功能仿真结果：



时序仿真结果：



4. 参照 sync.vhd 的代码，将 hello 的 BDF 图形用 VHDL 语言编写，并进行编译和仿真。

代码如下：

library ieee; --库声明部分：引入 IEEE 标准库中的必要包

```

use ieee.std_logic_1164.all; --标准逻辑库

use ieee.std_logic_arith.all; --算术运算库

use ieee.std_logic_unsigned.all; --无符号数运算库

--实体(Entity)声明： 定义模块的输入输出接口

entity hellovhdl is

    port

    (

        a, b, clk, nrst : in std_logic;

        d                : out std_logic

    );

end entity hellovhdl;

-- 架构(Architecture)声明： 描述模块的内部行为和结构， 架构名称为
hellovhdl_rtl, 属于实体 hellovhdl

architecture hellovhdl_rtl of hellovhdl is

    signal c : std_logic; --内部信号声明： 用于模块内部连接的信号

begin

    c <= a and b;

    process (clk, nrst) --进程(Process)声明： 描述时序逻辑

    begin

        if (nrst = '0') then --异步复位： 当 nrst 为低电平时执行复位

            d <= '0';

        elsif (rising_edge(clk)) then

```

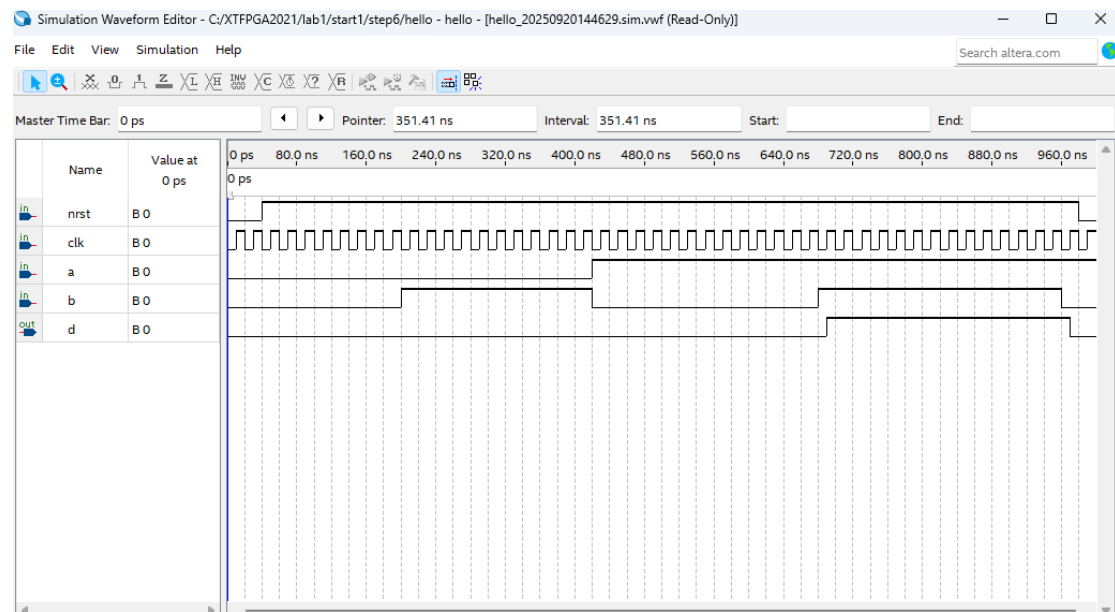
```
d <= c;
```

```
end if;
```

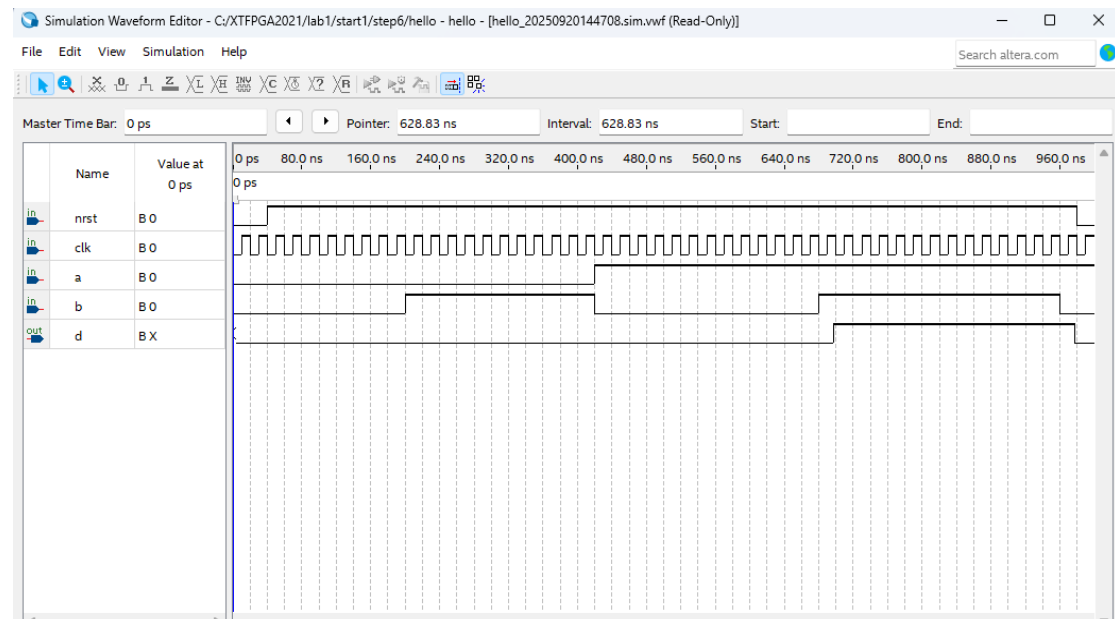
```
end process;
```

```
end architecture hellovhdl_rtl;
```

功能仿真结果：



时序仿真结果：



三、结果分析

Verilog HDL、VHDL 和 BDF 图形输入是数字电路设计中常用的三种设计输入方法。它们各有特点，但最终目的都是描述数字系统的结构和行为。

三者之间的区别：

1. BDF（图形输入）：

是一种图形化设计方法，使用逻辑符号通过连线表示电路结构。图形输入更接近物理结构，直观易懂，适合初学者理解电路连接，但不适合描述复杂行为或算法，也不适合直接进行仿真。

2. Verilog HDL：

是一种硬件描述语言，语法类似 C 语言，简洁灵活，适合算法和复杂逻辑的描述，支持并发和顺序语句，适合仿真和综合。使用 module 定义接口，always 块描述时序或组合逻辑。

3. VHDL：

也是一种硬件描述语言，语法相比 verilog 更严谨、冗长，强调类型安全和模块化，更适合大型系统设计和可靠性要求高的场合。使用 entity 定义接口，architecture 描述行为，强调类型和端口声明。

三者之间的联系：

三种方法都可以描述相同的数字逻辑功能，在描述同一个数字电路系统时本质是一致的，例如实验中的 D 触发器电路，用 BDF、Verilog、VHDL 三种方式实现的功能是完全一致，也是可以相互转化的。

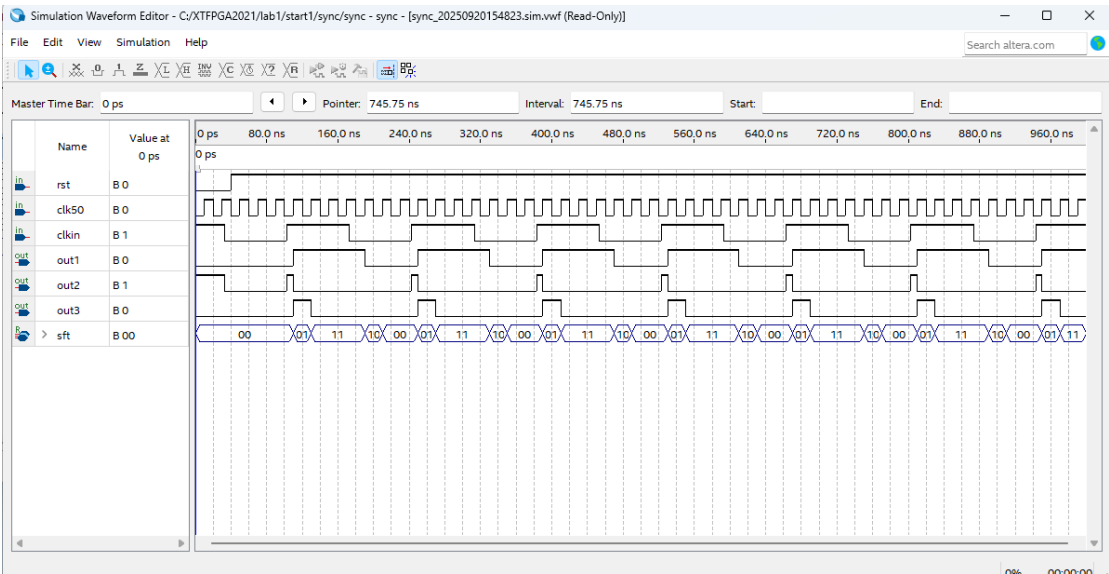
同时 Quartus 也支持在同一项目中混合使用 BDF、Verilog 和 VHDL，HDL 代码也可以表示为图形模块（Symbol）并在 BDF 中调用。例如：顶层用 BDF 连接

各个模块，模块内部用 Verilog 或 VHDL 实现。

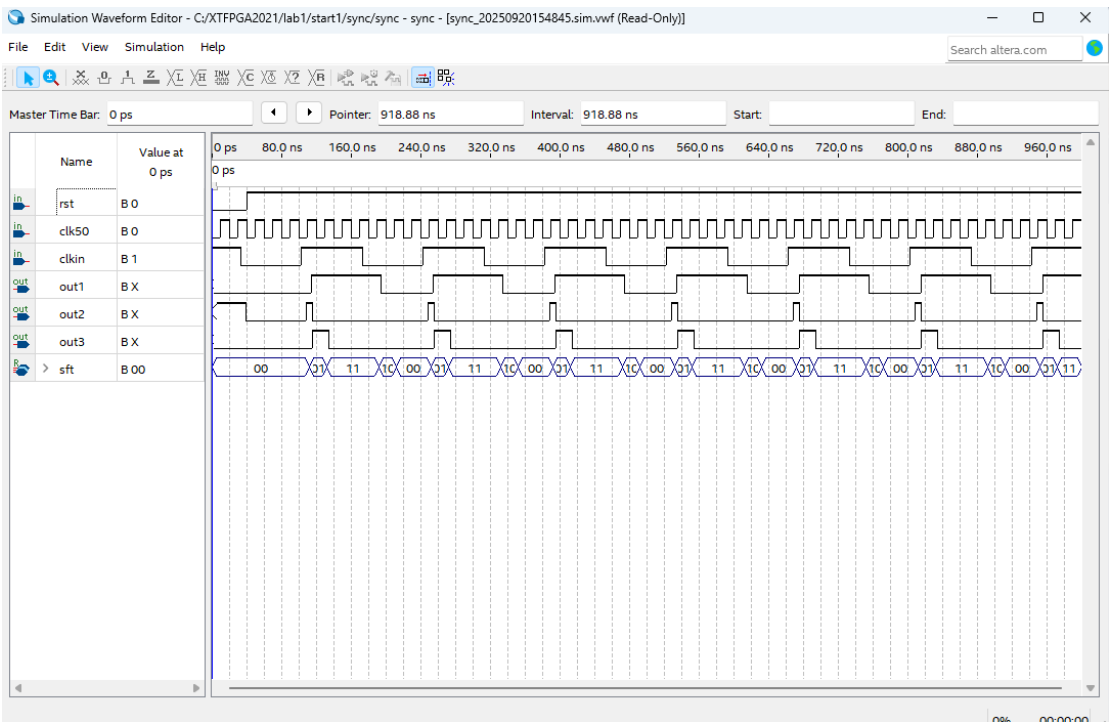
四、思考题

(1) 仿真 sync.vhd 并分析代码功能。

功能仿真结果：



时序仿真结果：



Out1 输出尝试解决输入信号怎么进入同步时序电路，Out3 尝试解决输入数


```
always @(posedge clk50 or negedge rst) begin
```

```
    if (!rst) begin
```

```
        sft0 <= 1'b0;
```

```
        sft1 <= 1'b0;
```

```
    end else begin
```

```
        sft0 <= clk50;
```

```
        sft1 <= sft0;
```

```
    end
```

```
    out1 = sft0;
```

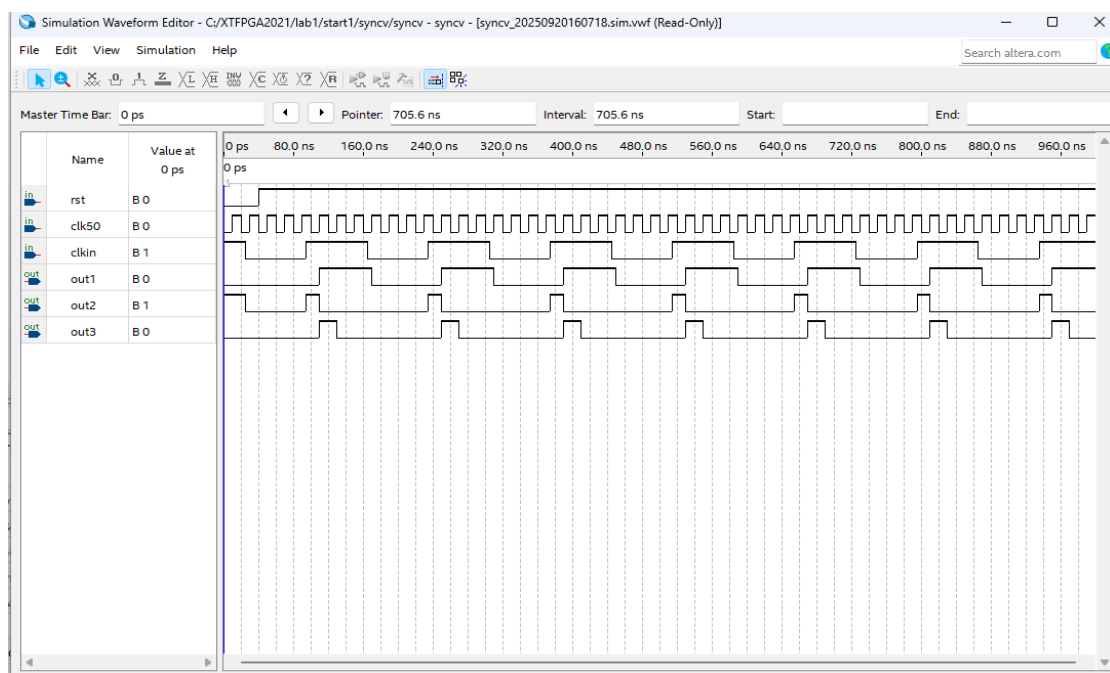
```
    out2 = ~sft0 & clk50;
```

```
    out3 = ~sft1 & sft0;
```

```
end
```

```
endmodule
```

功能仿真结果：



时序仿真结果：

